

OpsPilot AI - Sprint 2: Basic Incident Analysis

Goal: Analyse an incoming monitoring alert, detect whether its threshold is breached, and return a probable cause, recommended action, and confidence score.

What We Built

- Created a new **app/analyzer.py** module.
- Added rule-based probable causes and remediation recommendations.
- Compared alert values with thresholds.
- Connected the analyser to **POST /alerts**.
- Returned analysis with **201 Created**.

Current Flow

Alert received -> Validate alert -> Compare value and threshold -> Select probable cause -> Select recommended action -> Return analysis

Why We Created analyzer.py

The analysis logic is kept separate from the API code. **main.py** handles web requests, while **analyzer.py** handles incident analysis. This makes the project easier to understand, test, and maintain.

Rule Dictionaries

```
CAUSES = {
    "HIGH_CPU": "A resource-intensive process or sudden traffic increase",
    "HIGH_MEMORY": "A possible memory leak or excessive application workload",
    "DISK_FULL": "Rapid log growth, temporary files, or unused data",
    "SERVICE_DOWN": "Application crash, dependency failure, or configuration issue",
    "HTTP_5XX_SPIKE": "Backend exception or unavailable dependency"
}

ACTIONS = {
    "HIGH_CPU": "Inspect running processes and recent deployments",
    "HIGH_MEMORY": "Check memory-consuming processes and application logs",
    "DISK_FULL": "Remove unnecessary files and rotate old logs",
    "SERVICE_DOWN": "Check service status, logs, and dependent services",
    "HTTP_5XX_SPIKE": "Inspect application errors and dependency health"
}
```

How analyze_alert() Works

```
def analyze_alert(alert_type: str, value: float, threshold: float):
    is_breached = (
        value <= threshold
        if alert_type == "SERVICE_DOWN"
        else value >= threshold
    )
```

For CPU, memory, disk, and HTTP errors, a higher value means a problem. For SERVICE_DOWN, a lower health value can represent failure. Example: value 0 and threshold 1 means the service is down.

Concept	Meaning
Dictionary	Stores alert types as keys and their causes/actions as values.
Function	Reusable block of logic that analyses one alert.
Parameter	Input given to the function: alert_type, value, and threshold.
Boolean	True or False result stored in is_breached.
Conditional expression	Uses one comparison for SERVICE_DOWN and another for other alerts.
Return value	A dictionary containing the analysis result.

Response Logic

```
return {
    "investigation_status": "issue_detected" if is_breached else "normal",
    "probable_cause": CAUSES[alert_type] if is_breached else "No issue detected",
    "recommended_action": ACTIONS[alert_type] if is_breached else "Continue monitoring",
    "confidence": 85 if is_breached else 40
}
```

If the threshold is breached, the engine returns an issue, probable cause, action, and confidence 85. Otherwise, it returns normal status and recommends continued monitoring.

Connection with main.py

```
from app.analyzer import analyze_alert

analysis = analyze_alert(
    alert_type=alert.alert_type,
    value=alert.value,
    threshold=alert.threshold
)
```

main.py receives the alert and passes only the required values to analyzer.py. The returned analysis is then included in the API response.

Sample Result

```
{
  "investigation_status": "issue_detected",
  "probable_cause": "A resource-intensive process or sudden traffic increase",
  "recommended_action": "Inspect running processes and recent deployments",
  "confidence": 85
}
```

Important Limitation

This is currently a **rule-based analysis engine**, not Generative AI. The causes and actions are selected from predefined rules. This gives us a reliable foundation before adding logs, metrics, historical incidents, and AI-generated explanations.

Error and Learning

Problem: Swagger showed “Failed to fetch.”

Cause: The backend server was not reachable after the code change/reload.

Fix: Restart Uvicorn, wait for “Application startup complete,” refresh Swagger, and test again.

Judae Explanation

"In Sprint 2, we added a modular incident-analysis engine. It compares the incoming metric with its threshold and returns a probable cause, recommended action, and confidence score. We deliberately started with deterministic rules to provide reliable and explainable results before adding AI enrichment."

Quick Judge and Interview Questions

Q: Why separate analyzer.py from main.py?

A: To separate API handling from business logic and improve maintainability.

Q: Is this already AI?

A: No. It is a deterministic rule engine that will later be enriched with AI.

Q: Why start with rules?

A: Rules provide reliable, explainable, and testable results.

Q: How is an issue detected?

A: The alert value is compared with its configured threshold.

Q: Why is SERVICE_DOWN treated differently?

A: A lower health value can represent a failed service.

Q: What does confidence 85 mean?

A: It is a prototype confidence value for a matched rule, not a trained probability.

Sprint 2 Summary

OpsPilot AI can now receive an alert, detect a threshold breach, and return an explainable probable cause and recommended action.